

B+Tree Estructura y Propiedades

**Esteban Luna Seña
Elena Vargas Grisales
Juan Jose Paternina Cuava
Anderson David Rúa Escobar**

**Estructura de Datos
Universidad de Antioquia**



**UNIVERSIDAD
DE ANTIOQUIA**
1803

Contenido

01 Historia y contexto
¿De dónde viene el B+Tree?

02 El problema que resuelve
¿Por qué no basta un índice simple?

03 Del árbol binario al B+Tree
Evolución de estructuras

04 Definición y anatomía
Los tres tipos de nodos

05 Propiedades formales
Las reglas que garantizan el rendimiento

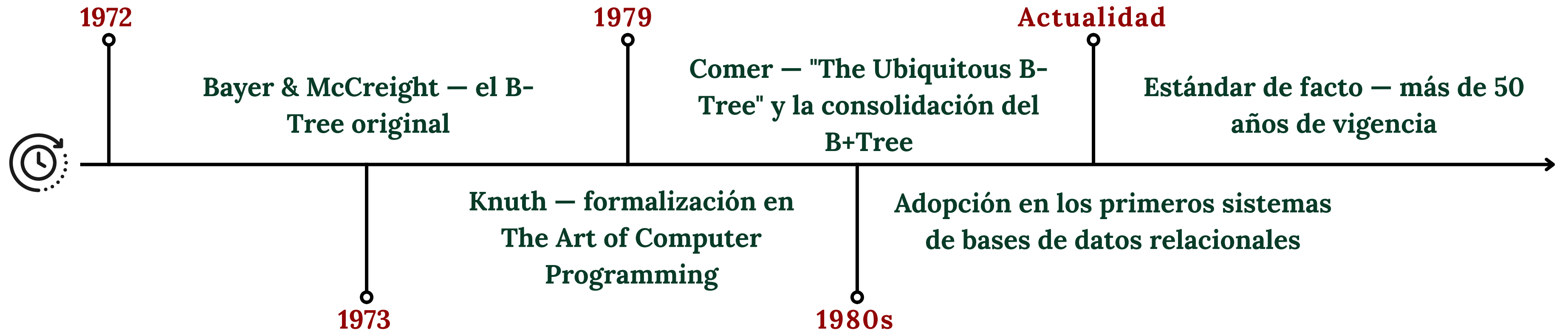
06 Ventajas-Desventajas
Comparación

07 Operaciones
Búsqueda puntual

08 Usos reales
PostgreSQL, MySQL, MongoDB...



Origen y evolución del B+Tree



El problema de acceso eficiente a datos en disco

- Un archivo de 10^8 registros de 100 bytes ocupa ~10 GB en disco
- Un scan secuencial requiere 10^6 lecturas de bloque (4 KB c/u)
→ inaceptable en tiempo de respuesta
- Búsqueda binaria sobre un índice denso: $O(\log_2 N)$ pero asume archivo ordenado estático
- Las inserciones y eliminaciones desordenan el archivo → necesitamos reorganización periódica costosa

10^6

I/O de bloque en scan completo

3-4

I/O con B+Tree (10^8 registros)



Contexto: Del árbol binario al B+Tree



Estructura	Balance	Branching factor	Búsqueda	Rango	Actualización dinámica
BST	No garantizado	2	$O(n)$ peor caso	$O(n)$	Sí
AVL / Red-Black	Garantizado	2	$O(\log_2 N)$	$O(\log N + k)$	Sí
B-Tree	Garantizado	$n \approx 100-600$	$O(\log n N)$	$O(\log n N + k)$ traversal complejo	Sí
B+Tree ✓	Garantizado	$n \approx 100-600$	$O(\log n N)$	$O(\log n N + k)$ scan lineal de hojas	Sí



Diferencia clave B-Tree → B+Tree: en B-Tree los datos pueden estar en nodos internos, lo que complica el range scan. En B+Tree, los datos están solo en hojas, que están enlazadas — el range scan es siempre un scan lineal de páginas contiguas.



¿Qué es un B+Tree?

Definición desde cero

Un B+Tree es un árbol de búsqueda con estas dos ideas fundamentales:

- ① Todos los datos reales están guardados en las hojas
- ② Las hojas están conectadas entre sí como una cadena de nodos

Nodo Raíz

Es el punto de entrada.
Siempre hay exactamente uno.
Desde aquí comienza toda búsqueda.

Nodos Internos

Son las 'señales de tráfico'.
No guardan datos, solo claves que guían la búsqueda hacia abajo.

Nodos Hoja

Aquí viven los datos reales (o punteros a los registros).
Están enlazados en orden.

La 'B' de B+Tree viene del inglés balanced (equilibrado) – todas las hojas están a la misma profundidad.

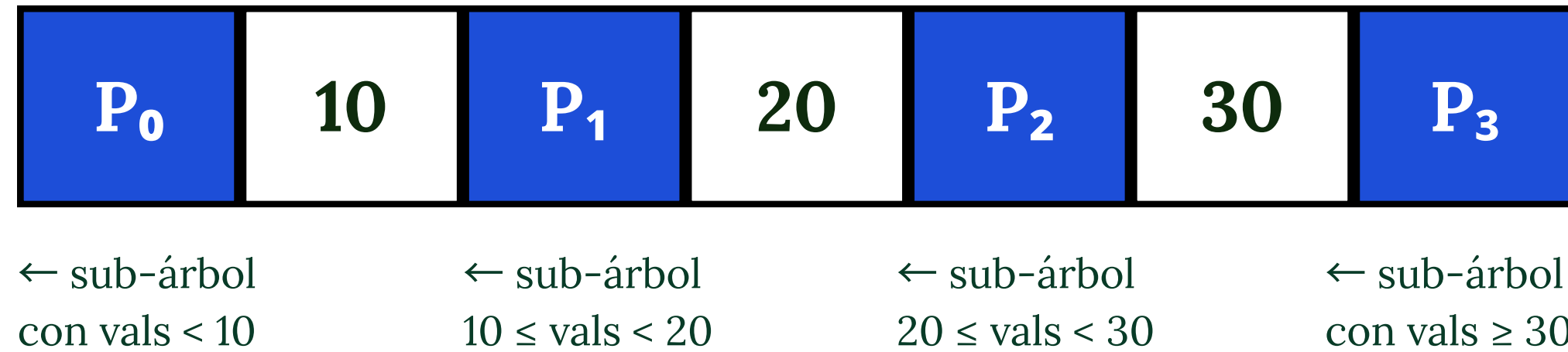


Anatomía del Nodo Interno

Estructura formal:

$P_0 \mid K_1 \mid P_1 \mid K_2 \mid P_2 \mid \dots \mid K_{n-1} \mid P_{n-1}$

P_i = puntero a sub-árbol · K_i = clave guía



Reglas de llenado (n = orden del árbol):

Nodo interno (no raíz): mínimo $\lceil n/2 \rceil$ punteros, máximo n punteros | Raíz: mínimo 2 punteros (salvo árbol de un solo nodo)

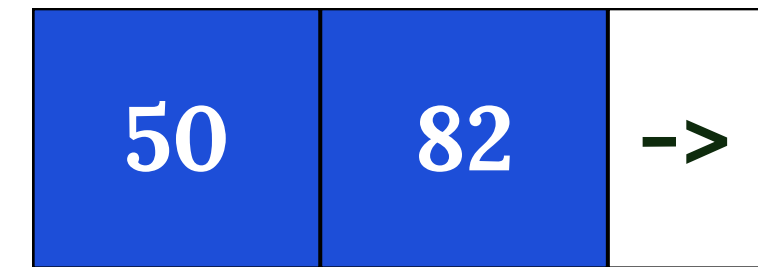
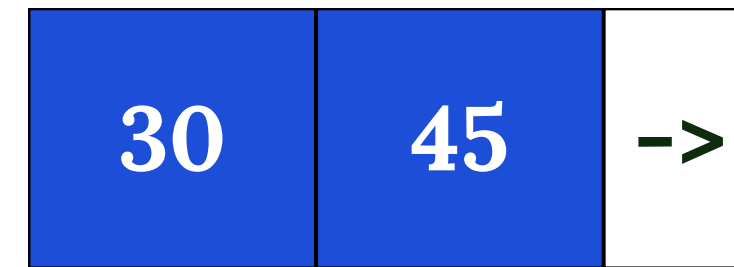
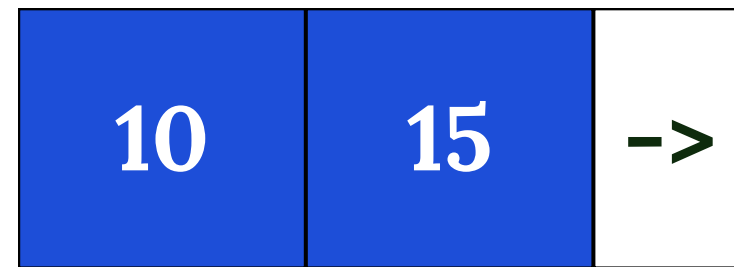
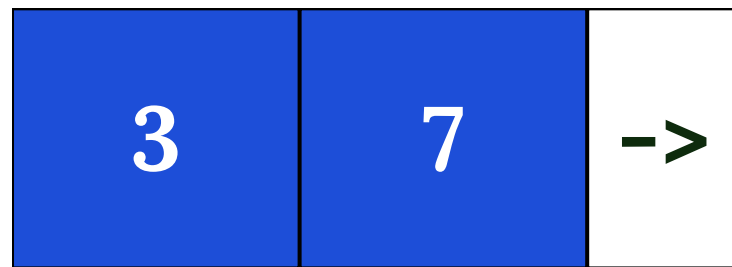


Anatomía del Nodo Hoja

Estructura formal:

$(K_1, ptr_1) \mid (K_2, ptr_2) \mid \dots \mid (K_{n-1}, ptr_{n-1}) \mid P_{next}$

$ptr_i \rightarrow$ registro real en el archivo · $P_{next} \rightarrow$ siguiente hoja



Lista enlazada ordenada $\rightarrow$ recorre las hojas de izquierda a derecha para obtener todos los datos en orden ascendente

Reglas del nodo hoja:

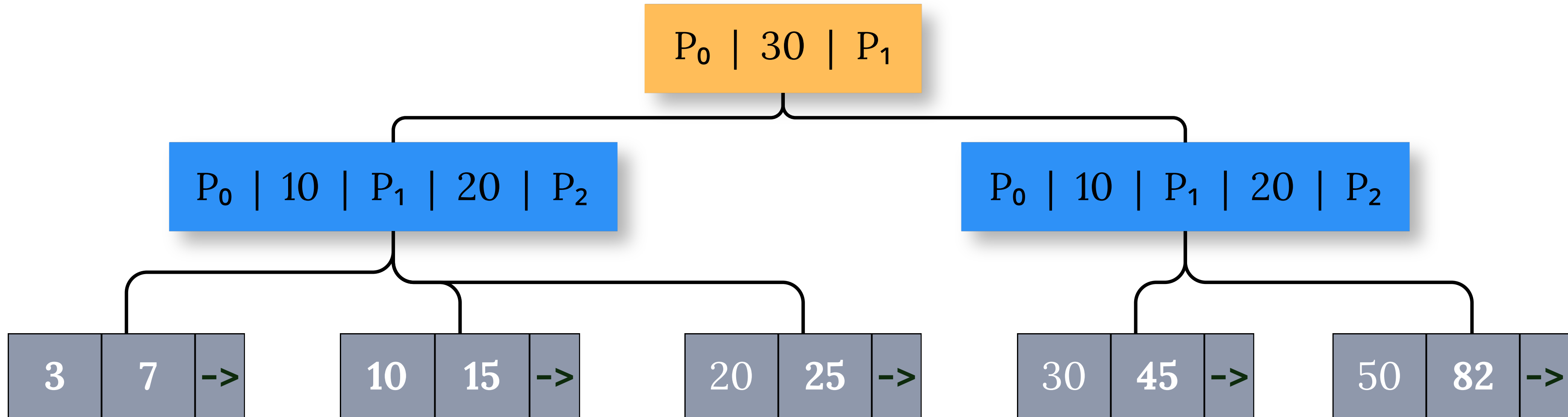
Cada hoja guarda hasta $n-1$ valores de clave · mínimo $\lceil (n-1)/2 \rceil$ valores

- Todos los datos reales están aquí (índice denso)
- P_{next} encadena las hojas en orden ascendente $\rightarrow$ permite range scans eficientes



El B+Tree Completo — Visión Global

Ejemplo con $n = 3$ (máx. 2 claves por nodo)



Nodo raíz

Nodo interno

Nodo hoja

← Lista enlazada de hojas (en orden): $3 \rightarrow 7 \rightarrow 10 \rightarrow 15 \rightarrow 20 \rightarrow 25 \rightarrow 30 \rightarrow 45 \rightarrow 50 \rightarrow 82$



Las 6 Propiedades Formales del B+Tree



Las reglas que garantizan el rendimiento

01 Hojas a igual profundidad

Todas las hojas están al mismo nivel h . Garantiza que toda búsqueda tiene el mismo costo.

02 Nodo interno (no raíz)

Tiene entre $\lceil n/2 \rceil$ y n hijos. Mínimo 50% lleno. Garantiza que el árbol no se vuelva demasiado alto.

03 Nodo hoja

Tiene entre $\lceil (n-1)/2 \rceil$ y $n-1$ entradas. También mínimo 50% lleno.

04 Raíz

Tiene entre 2 y n hijos (si no es hoja). Es la única excepción al llenado mínimo.

05 Datos solo en hojas

Los registros reales (o punteros a ellos) viven solo en los nodos hoja. Los nodos internos solo tienen claves guía.

06 Hojas enlazadas

Las hojas forman una lista enlazada ordenada. Permite recorrerlas en orden sin volver a la raíz.



Las 3 Invariantes: Garantías del B+Tree



Propiedades que NUNCA se rompen

- 1 Balance perfecto: todas las hojas a idéntica profundidad h
- 2 Llenado mínimo $\geq 50\%$: cada nodo (no raíz) tiene al menos $\lceil n/2 \rceil$ hijos
- 3 Orden global: la lista enlazada de hojas cubre todas las claves en orden ascendente

n	N registros	Altura h
100	10^6	3
200	10^8	4
585	10^9	4



Propiedades: Altura y Capacidad



¿Por qué el B+Tree es tan eficiente?

Altura máxima garantizada:

$$h \leq \lceil \log_{(n/2)}(N) \rceil$$

N = número de registros

n = orden del árbol (hijos por nodo)

Con página de 8 KB y claves int8:

$n \approx 585 \rightarrow \lceil \log_{292}(10^8) \rceil = 4$
niveles

Capacidad por nodo (página 8 KB):

Nodo interno: $n \approx 585$ punteros

Nodo hoja: $n-1 \approx 511$ entradas

fillfactor=90% en PostgreSQL
(deja 10% libre para inserciones
sin tener que dividir el nodo)

¿Por qué importa? → Con $n=585$, la raíz apunta hasta 585 sub-árboles. Cada sub-árbol apunta hasta 585 más. En solo 4 niveles caben 10^8 registros. Cada nivel = 1 lectura de disco. Resultado: 4 lecturas totales.

n	N registros	Altura h	Lecturas de disco
100	10^6	3	3
200	10^8	4	4
585	10^9	4	4



Operación: Búsqueda Puntual

Buscar $k = 25$ paso a paso



1

Raíz [30]

$25 < 30 \rightarrow$ bajar por P_0 (rama izquierda)

2

Nodo interno [10 | 20]

$20 \leq 25 \rightarrow$ bajar por P_2 (segunda rama derecha)

3

Hoja [20, 25]

Scan lineal en la hoja $\rightarrow$ 25 encontrado en posición 1

Costo exacto:

h lecturas de disco

$h=4, n=585, N=10^8 \rightarrow$ 4 lecturas de bloque
(comparado con 10^6 sin índice)

Búsqueda por rango:

WHERE k BETWEEN 10 AND 45

1. Bajar al nodo hoja que contiene 10
2. Seguir los punteros P_{next} de hoja en hoja hasta agotar el rango
3. Costo: $O(\log_n N + k)$

```
find(nodo, clave): si nodo.es_hoja  $\rightarrow$  buscar clave en nodo.claves  $\rightarrow$  retornar puntero (o None si no existe)
                  sino  $\rightarrow$  encontrar el índice  $i$  tal que  $clave < \text{nodo.claves}[i] \rightarrow$  recurrir en  $\text{find}(\text{nodo.hijos}[i], \text{clave})$ 
```



Ventajas y Desventajas



Ventajas

- Uso eficiente del disco (menos I/O)
- Búsquedas de rango muy rápidas
- Árbol siempre balanceado ($O(\log n)$)
- Alta ramificación → menor altura
- Acceso secuencial eficiente (hojas enlazadas)
- Ideal para índices en MySQL y PostgreSQL



Desventajas

- Implementación compleja
- Mayor uso de memoria
- Mantenimiento costoso
- No optimo para búsquedas exactas simples
- Posible desperdicio de espacio
- Dependencia del tamaño del bloque



Aplicaciones En La Indsutria

amazon



Uber



El B+Tree es el índice más usado en DBMS



PostgreSQL

Usa B+Tree por defecto para CREATE INDEX.
fillfactor=90 reduce splits en inserciones.



MySQL /
InnoDB

El índice primario (clustered index) ES el B+Tree.
Las hojas almacenan filas completas.



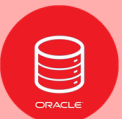
SQLite

Toda la BD es un único B+Tree.
Las tablas e índices son sub-árboles.



Oracle DB

Índice estándar. Soporta variantes como Bitmap
Index para columnas de baja cardinalidad.



MongoDB

Usa WiredTiger con B+Trees para índices. Las
colecciones también usan B+Trees internamente.



Sistemas de
archivos

NTFS (Windows), HFS+ y APFS (macOS) y ext4
(Linux) usan B+Trees para directorios.



Caso de estudio



Contexto y Problema

Sector bancario de Vietnam · Datos de transacciones masivos

Cluster Comput (2019) 22:S1011–S1021
<https://doi.org/10.1007/s10586-017-1183-y>



B+-tree construction on massive data with Hadoop

Huynh Cong Viet Ngu¹ · Jun-Ho Huh²

Received: 14 May 2017 / Revised: 2 August 2017 / Accepted: 11 September 2017 / Published online: 21 September 2017
 © Springer Science+Business Media, LLC 2017

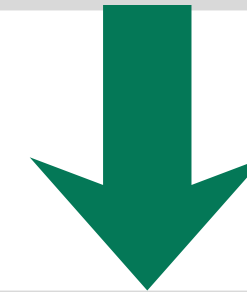
FPT University, Vietnam · Catholic University of Pusan, Korea

Cluster Computing · Springer, 2019

DOI: 10.1007/s10586-017-1183-y

CONTEXTO

- Vietnam enfrenta un rápido crecimiento económico y aumento explosivo en transacciones bancarias diarias.
- La Ley 07/2012/QH13 contra el lavado de dinero exige monitoreo continuo de historial de clientes.
- El volumen de datos supera la capacidad de los sistemas de indexación tradicionales.
- Se necesita una tecnología eficiente para detectar transacciones sospechosas en tiempo real.



PROBLEMA

+172 Millones de datos / 10GB transacciones procesadas en el experimento





Fundamento teórico de la solución propuesta

Auto-balanceo

Mantiene todos los nodos hoja al mismo nivel. Garantiza altura logarítmica $O(\log n)$ independientemente del volumen de datos.

Datos en las hojas

Solo los nodos hoja almacenan registros completos. Los nodos internos contienen únicamente claves pivot para guiar la búsqueda.

Hojas encadenadas

Las hojas están conectadas en lista enlazada, lo que habilita consultas de rango secuenciales ultra-eficientes.

¿Por qué B+Tree es ideal para el sector bancario?

- Datos ordenados → el historial de consultas de las transacciones era ordenado en poco tiempo.
- Consultas de rango → se realizaba la extracción de períodos completos de historial bancario de manera rápida y sin pérdida de información.
- Búsqueda en $O(\log n)$ → se usaba para la localización rápida de transacciones sospechosas
- Optimizado para disco → mínimo de operaciones I/O en grandes volúmenes



Caso de estudio

Solución Propuesta: Sistema Paralelo B+Tree + Hadoop



Tres fases ejecutadas con el modelo MapReduce

01 Distribución de Datos

- M Mappers leen muestras del conjunto de datos (10% o 100%).
- Un único Reducer determina R-1 fronteras de partición.
- Cada partición contiene igual número de transacciones.
- Clave usada: o.d (fecha/hora de transacción)

02 Construcción B+Trees

- Fase más intensiva computacionalmente del esquema.
- Cada Reducer construye un B+Tree local en paralelo.
- Técnica bottom-up bulk loading maximiza nodos hoja.
- Minimiza la altura de cada árbol → mejor desempeño.

03 Combinación final

- Los R B+Trees pequeños se unen bajo un único nodo raíz.
- Lógica simple → no requiere ejecutarse en el clúster.
- Produce un B+Tree completo sobre todo el dataset.
- Permite consultas de rango sobre millones de registros.

El modelo Hadoop MapReduce garantiza: distribución automática · tolerancia a fallos · escalabilidad horizontal



Caso de estudio

Resultados Experimentales

Clúster de 8 nodos CentOS · Hadoop 2.7.2 · 3 tamaños de datos

6 GB

98.8 M transacciones

8 GB

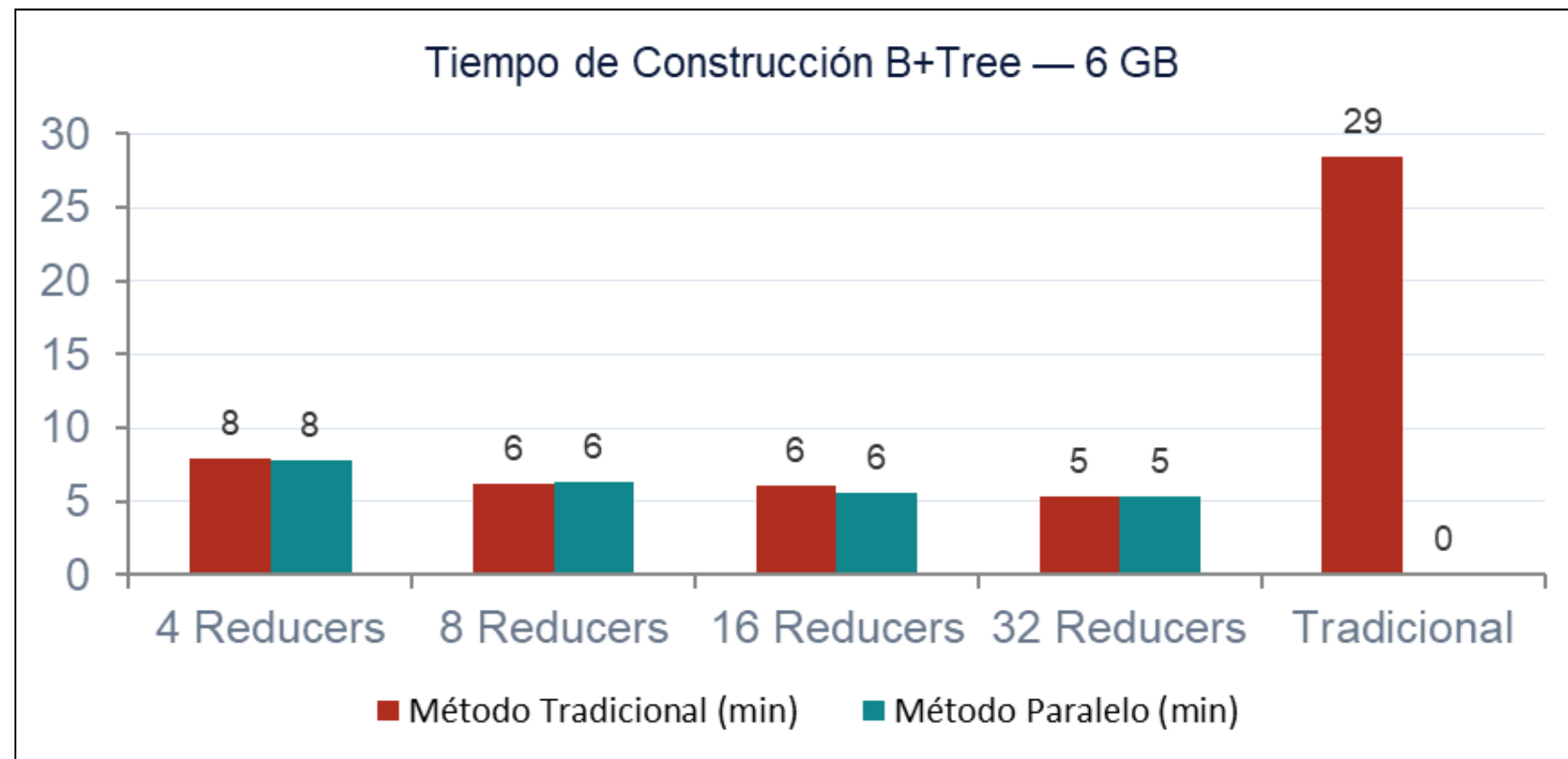
138.3 M transacciones

10 GB

172.9 M transacciones

Hallazgos Clave

- ~2× más rápido que el método tradicional (con 4 reducers).
- +Reducers el tiempo disminuye al agregar nodos de cómputo.
- 10% sample suficiente para determinar fronteras de partición.
- Mejor calidad bulk loading minimiza altura → consultas más rápidas.



Caso de estudio

Conclusiones y Aporte Industrial

Impacto bancario directo

El sistema permite monitorear millones de transacciones continuas, facilitando el cumplimiento de la Ley anti-lavado de dinero 07/2012.

Escalabilidad demostrada

Procesó hasta 172.9 M transacciones (10 GB). Al aumentar reducers, el tiempo de construcción decrece linealmente.

Propiedades B+Tree aprovechadas

El bulk-loading bottom-up maximiza nodos hoja y minimiza la altura, optimizando las propiedades fundamentales de la estructura.

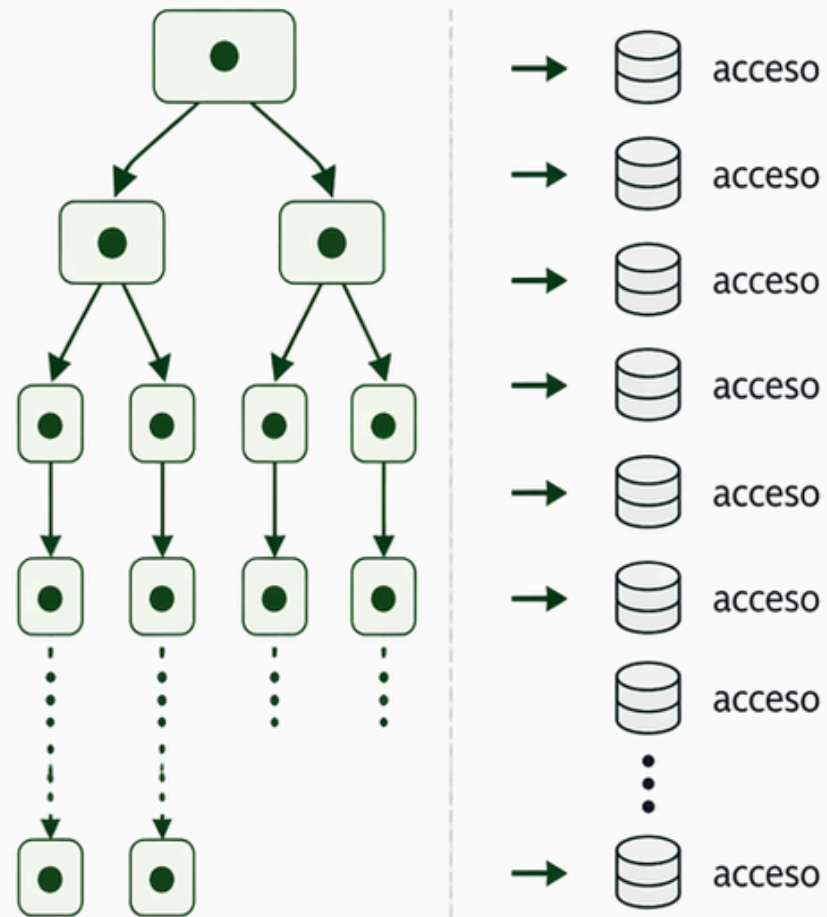
Aporte a Cloud Computing

La arquitectura es aplicable a cualquier sistema bancario distribuido que use Hadoop/HDFS como infraestructura de datos.



¿Por qué el B+Tree es eficiente en disco?

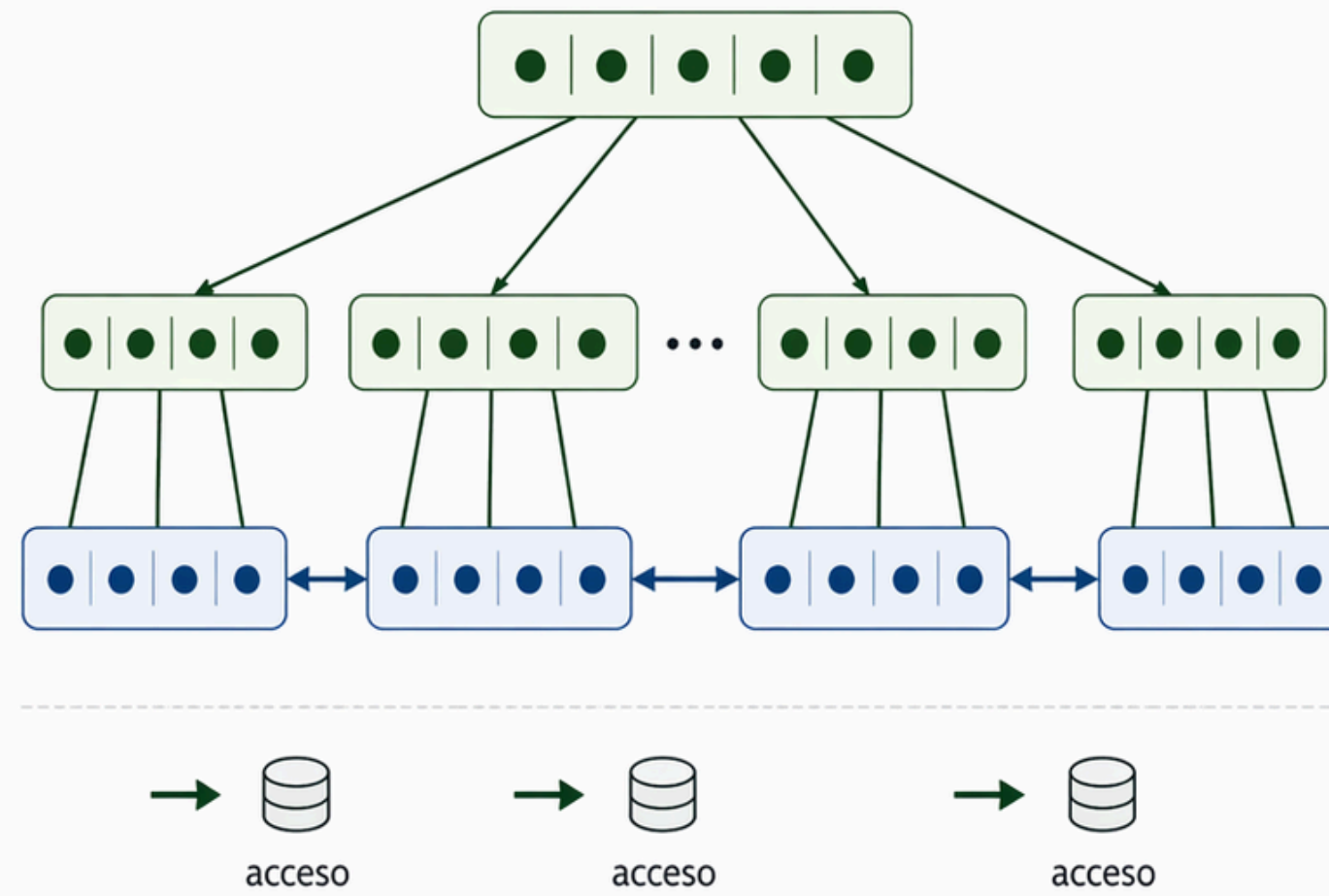
Árbol binario



Poca ramificación

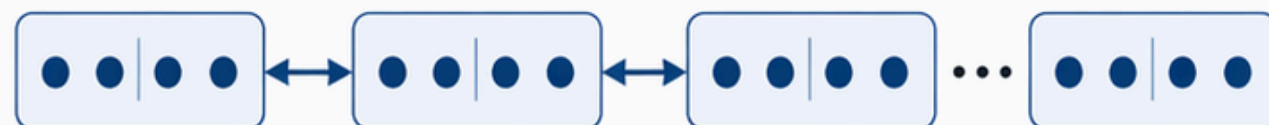
Más altura = más accesos a disco

B+Tree



Alta ramificación

Menor altura = menos accesos a disco



Hojas enlazadas → consultas por rango eficientes

- Cada nodo suele almacenarse en una página de disco
- En una sola página caben muchas claves y punteros
- Esto le da al árbol una alta ramificación
- **Alta ramificación = menor altura**
- **Menor altura = menos accesos a disco**
- Además, sus hojas enlazadas permiten consultas por rango eficientes



Conclusión

- 01 El B+Tree resuelve el problema de acceso eficiente a disco**
pasando de 10^6 lecturas a solo 3-4 lecturas para 10^8 registros.
- 02 Su estructura de 3 tipos de nodos + hojas enlazadas**
permite tanto búsquedas puntuales $O(\log_n N)$ como rangos $O(\log_n N + k)$.
- 03 Sus 3 invariantes (balance, llenado $\geq 50\%$, orden global)**
garantizan rendimiento predecible en todo momento.
- 04 Es el estándar de facto desde hace 50 años**
presente en PostgreSQL, MySQL, SQLite, Oracle, MongoDB y más.



Referencias

- Geeks Forgeeks. (18 de abril de 2026). <https://www.geeksforgeeks.org/>. Recuperado el 19 de abril de 2026, de <https://goo.su/IVVc7>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (16 de agosto de 2006). <https://elhacker.info>. Recuperado el 19 de abril de 2026, de <https://goo.su/oGu0o>
- WsCUBE TECH. (26 de febrero de 2026). <https://www.wscubetech.com/>. Recuperado el 19 de abril de 2026, de <https://goo.su/ExUog4>
- Ngu, H.C.V., Huh, JH. B+-tree construction on massive data with Hadoop. Cluster Comput 22 (Suppl 1), 1011–1021 (2019). <https://doi.org/10.1007/s10586-017-1183-y>

